

Лабораторная работа №4

Реализация основных моделей в агентном подходе.

Разанацуа Сара Естэлл

Российский университет дружбы народов, Москва, Россия

.. ..
.. ..

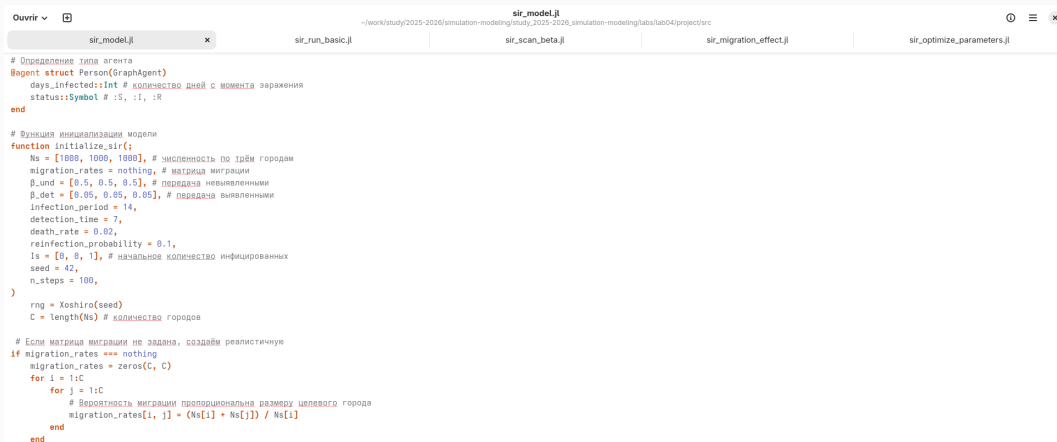
Создадим агентную модель распространения инфекционного заболевания на основе классической компартментальной модели SIR (Susceptible-InfectiousRecovered). Модель будет реализована с использованием пакета Agents.jl. В отличие от классической модели на дифференциальных уравнениях, агентный подход позволит учесть индивидуальные характеристики, пространственную структуру и стохастичность процессов.

Задание

1. Создать рабочий каталог для кода.
2. Установить необходимые пакеты.
3. Выполнить предложенный код.
4. Преобразовать код в литературный стиль.
5. Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
6. Выполнить код из jupyter notebook.
7. Интегрировать документацию в формате Quarto в отчёт.
8. Добавить в код в литературном стиле вычисление для набора параметров.
9. Сгенерировать из литературного кода с параметрами:
 - чистый код;

Выполнение лабораторной работы

- Мы создадим файл `src/sir_model.jl` и запустим его.



```

sir_model.jl
~/work/study/2025-2026/simulation-modeling/study_2025-2026_simulation-modeling/labs/lab04/project/src

sir_model.jl x sir_run_basic.jl sir_scan_beta.jl sir_migration_effect.jl sir_optimize_parameters.jl

# Определение типа агента
@agent struct Person(GraphAgent)
    days_infected::Int # количество дней с момента заражения
    status::Symbol # :S, :I, :R
end

# Функция инициализации модели
function initialize_sir(;
    Ns = [1000, 1000, 1000], # численность по трём городам
    migration_rates = nothing, # матрица миграции
    β_und = [0.5, 0.5, 0.5], # передача невыявленными
    β_det = [0.05, 0.05, 0.05], # передача выявленными
    infection_period = 14,
    detection_time = 7,
    death_rate = 0.02,
    reinfection_probability = 0.1,
    Is = [0, 0, 1], # начальное количество инфицированных
    seed = 42,
    n_steps = 100,
)
    rng = Xoshiro(seed)
    C = length(Ns) # количество городов

    # Если матрица миграции не задана, создаём реалистичную
    if migration_rates === nothing
        migration_rates = zeros(C, C)
        for i = 1:C
            for j = 1:C
                # Вероятность миграции пропорциональна размеру целевого города
                migration_rates[i, j] = (Ns[i] + Ns[j]) / Ns[i]
            end
        end
    end
end
```

- Вывод в консоль.

```
julia> include("src/sir_model.jl")  
total_count (generic function with 1 method)
```

Рис. 2: Рис

Базовый эксперимент

- Запускаем один эксперимент с фиксированными параметрами (по умолчанию) и сохраняем динамику численности агентов. Это служит для проверки работоспособности модели и получения базового понимания эпидемического процесса.

```
Ouvrir ▾  sir_run_basic.jl ~/work/study/2025-2026/simulation-modeling/study_2025-2026_simulation-modeling/labs/lab04/project/scripts   
sir_model.jl sir_run_basic.jl x sir_scan_beta.jl sir_migration_effect.jl sir_optimize_parameters.jl

# Параметры эксперимента
params = Dict{
    :Ns => [1000, 1000, 1000],
    :beta_und => [0.5, 0.5, 0.5],
    :beta_det => [0.05, 0.05, 0.05],
    :infection_period => 14,
    :detection_time => 7,
    :death_rate => 0.02,
    :reinfection_probability => 0.1,
    :Is => [0, 0, 1],
    :seed => 42,
    :n_steps => 100,
}

# Инициализация модели
model = initialize_sir(; params...)

# Подготовка массивов для хранения данных
times = Int[]
S_vals = Int[]
I_vals = Int[]
R_vals = Int[]
total_vals = Int[]

# Запуск симуляции вручную
for step = 1:params[:n_steps]
    Agents.step!(model, 1)

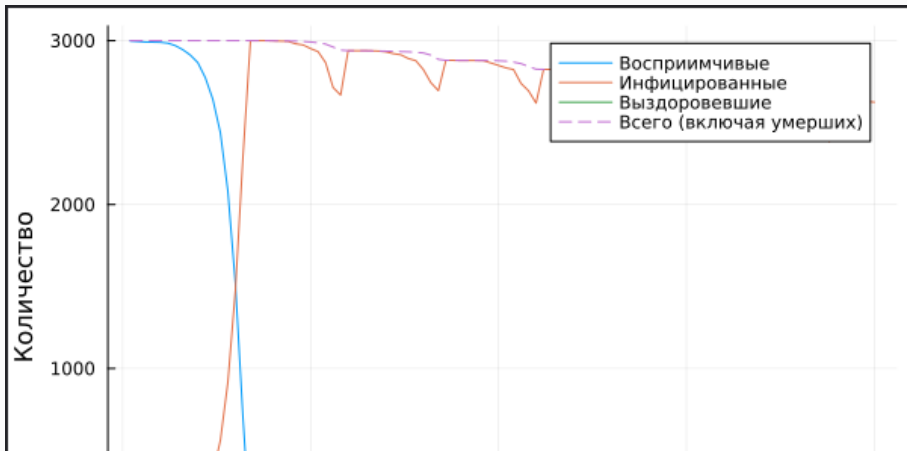
    push!(times, step)
    push!(S_vals, susceptible_count(model))
    push!(I_vals, infected_count(model))
    push!(R_vals, recovered_count(model))
end
```


- Мы создадим файл `src/sir_run_basic.jl` и запустим его.

```
julia> touch("scripts/sir_scan_beta.jl")
"scripts/sir_scan_beta.jl"

julia> include("scripts/sir_scan_beta.jl")
Завершён эксперимент с beta = 0.1, seed = 42
Завершён эксперимент с beta = 0.1, seed = 43
Завершён эксперимент с beta = 0.1, seed = 44
Завершён эксперимент с beta = 0.2, seed = 42
Завершён эксперимент с beta = 0.2, seed = 43
Завершён эксперимент с beta = 0.2, seed = 44
Завершён эксперимент с beta = 0.3, seed = 42
Завершён эксперимент с beta = 0.3, seed = 43
Завершён эксперимент с beta = 0.3, seed = 44
Завершён эксперимент с beta = 0.4, seed = 42
Завершён эксперимент с beta = 0.4, seed = 43
Завершён эксперимент с beta = 0.4, seed = 44
Завершён эксперимент с beta = 0.5, seed = 42
Завершён эксперимент с beta = 0.5, seed = 43
Завершён эксперимент с beta = 0.5, seed = 44
Завершён эксперимент с beta = 0.6, seed = 42
Завершён эксперимент с beta = 0.6, seed = 43
Завершён эксперимент с beta = 0.6, seed = 44
Завершён эксперимент с beta = 0.7, seed = 42
Завершён эксперимент с beta = 0.7, seed = 43
Завершён эксперимент с beta = 0.7, seed = 44
Завершён эксперимент с beta = 0.8, seed = 42
Завершён эксперимент с beta = 0.8, seed = 43
Завершён эксперимент с beta = 0.8, seed = 44
Завершён эксперимент с beta = 0.9, seed = 42
Завершён эксперимент с beta = 0.9, seed = 43
Завершён эксперимент с beta = 0.9, seed = 44
```

- График: `plots/sir_basic_dynamics.png` — четыре линии: $S(t)$, $I(t)$, $R(t)$ и общая численность. Позволяет визуально оценить пик эпидемии, скорость распространения и влияние смертности.



Сканирование коэффициента заразности

- Исследуем, как изменение базовой заразности (β_{und} и пропорционально β_{det}) влияет на эпидемические показатели: пик заболеваемости, долю переболевших, число умерших. Выполняется параметрическое сканирование с несколькими повторными прогонами для учёта стохастичности.

```

Ouvrir v [icon] sir_scan_beta.jl ~/work/study/2025-2026/simulation-modeling/study_2025-2026/simulation-modeling/labs/lab04/project/scripts
sir_model.jl sir_run_basic.jl sir_scan_beta.jl x sir_migration_effect.jl sir_optimize_parameters.jl

# Функция для запуска одного эксперимента и возврата метрик
function run_experiment(p)
    # Создаём  $\beta_{und}$  и  $\beta_{det}$  на основе скалярного beta
    beta = p[:beta]
     $\beta_{und}$  = fill(beta, 3)
     $\beta_{det}$  = fill(beta/10, 3)
    # Передаём в модель
    model = initialize_sir(;
        Ns = p[:Ns],
         $\beta_{und}$  =  $\beta_{und}$ ,
         $\beta_{det}$  =  $\beta_{det}$ ,
        infection_period = p[:infection_period],
        detection_time = p[:detection_time],
        death_rate = p[:death_rate],
        reinfection_probability = p[:reinfection_probability],
        Is = p[:Is],
        seed = p[:seed],
        n_steps = p[:n_steps], # этот параметр не используется в initialize_sir, но может быть нужен для цикла
    )

    infected_fraction(model) =
        count(a.status == :I for a in allagents(model)) / nagents(model)
    peak_infected = 0.0

    for step = 1:p[:n_steps]
        # Ручной шаг (безопасный)
        agent_ids = collect(allids(model))

        for id in agent_ids

```

Сканирование коэффициента заразности

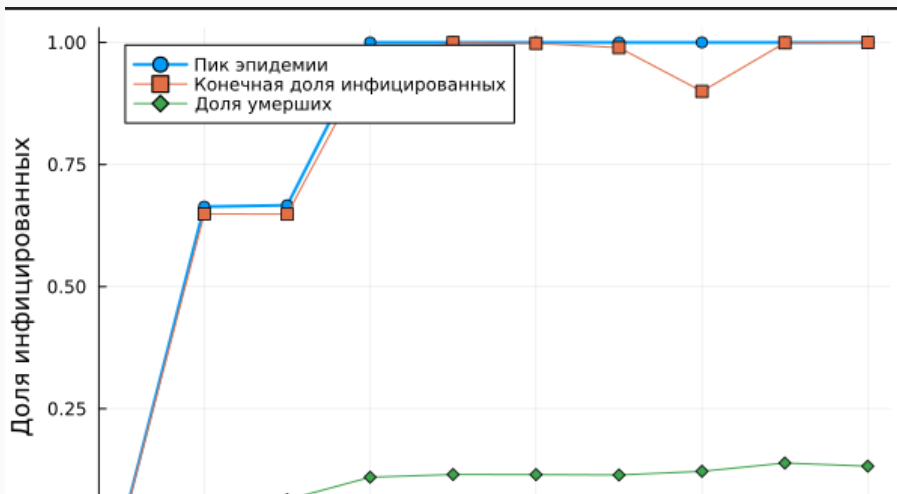
- Создание файла `scripts/sir_scan_beta.jl` и запустим его.

```
julia> touch("scripts/sir_migration_effect.jl")
"scripts/sir_migration_effect.jl"

julia> include("scripts/sir_migration_effect.jl")
Завершён эксперимент с migration_intensity = 0.0, seed = 42
Завершён эксперимент с migration_intensity = 0.0, seed = 43
Завершён эксперимент с migration_intensity = 0.0, seed = 44
Завершён эксперимент с migration_intensity = 0.1, seed = 42
Завершён эксперимент с migration_intensity = 0.1, seed = 43
Завершён эксперимент с migration_intensity = 0.1, seed = 44
Завершён эксперимент с migration_intensity = 0.2, seed = 42
Завершён эксперимент с migration_intensity = 0.2, seed = 43
Завершён эксперимент с migration_intensity = 0.2, seed = 44
Завершён эксперимент с migration_intensity = 0.3, seed = 42
Завершён эксперимент с migration_intensity = 0.3, seed = 43
Завершён эксперимент с migration_intensity = 0.3, seed = 44
Завершён эксперимент с migration_intensity = 0.4, seed = 42
Завершён эксперимент с migration_intensity = 0.4, seed = 43
Завершён эксперимент с migration_intensity = 0.4, seed = 44
Завершён эксперимент с migration_intensity = 0.5, seed = 42
Завершён эксперимент с migration_intensity = 0.5, seed = 43
```

Сканирование коэффициента заразности

- График позволяет найти пороговое значение β , при котором возникает эпидемия, и оценить, как увеличивается нагрузка на систему с ростом заразности



Исследование эффекта миграции

- Исследует, как интенсивность перемещения людей между городами влияет на скорость распространения эпидемии (время достижения пика) и масштаб пика. Инфекция начинается только в одном городе, остальные изначально здоровы.

```

sir_migration_effect.jl
~/work/study/2025-2026/simulation-modeling/study_2025-2026_simulation-modeling/labs/lab04/project/scripts

sir_model.jl | sir_run_basic.jl | sir_scan_beta.jl | sir_migration_effect.jl x | sir_optimize_parameters.jl

# Функция для создания матрицы миграции с заданной интенсивностью
function create_migration_matrix(C, intensity)
    M = ones(C, C) .* intensity ./ (C-1)
    for i = 1:C
        M[i, i] = 1 - intensity
    end
    return M
end

# Функция для измерения времени достижения пика
function peak_time(p)
    # Создаём матрицу миграции на основе интенсивности
    migration_rates = create_migration_matrix(p[:C], p[:migration_intensity])
    model = initialize_sir(;
        Ns = p[:Ns],
        β_und = p[:β_und],
        β_det = p[:β_det],
        infection_period = p[:infection_period],
        detection_time = p[:detection_time],
        death_rate = p[:death_rate],
        reinfection_probability = p[:reinfection_probability],
        Is = p[:Is],
        seed = p[:seed],
        migration_rates = migration_rates,
    )

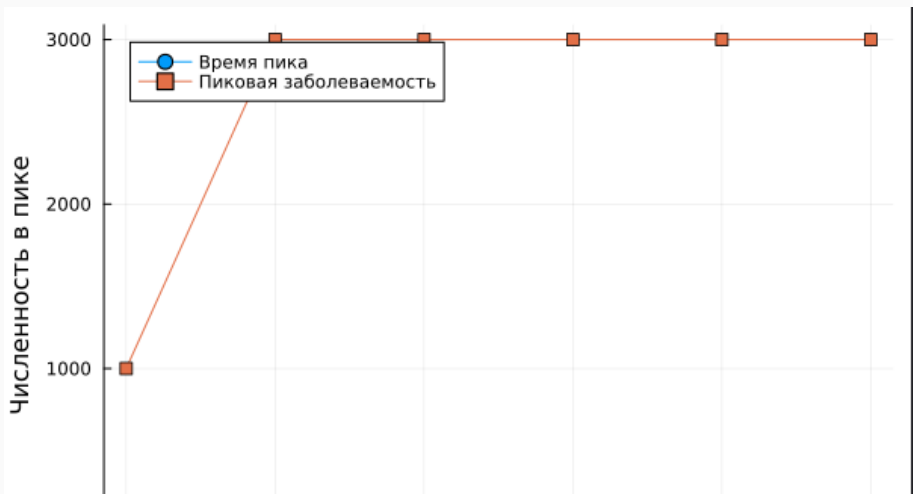
    infected_frac(model) = count(a.status == :I for a in allagents(model)) / nagents(model)
    peak = 0.0
    peak_step = 0

    for step = 1:p[:n_steps]
        # Ручной шаг
        agent_ids = collect(allids(model))

```

Исследование эффекта миграции

- График демонстрирует, как ускорение обмена людьми между городами приводит к более раннему и более высокому пику эпидемии.



Многокритериальная оптимизация параметров

- Ищем оптимальные комбинации параметров, минимизирующие одновременно два критерия: пиковую заболеваемость и долю умерших. Используем эволюционный алгоритм (Borg MOEA) из пакета BlackBoxOptim.

```

sir_optimize_parameters.jl
~/work/study/2025-2026/simulation-modeling/study_2025-2026/simulation-modeling/labs/lab04/project/scripts

sir_model.jl | sir_run_basic.jl | sir_scan_beta.jl | sir_migration_effect.jl | sir_optimize_parameters.jl x

# Целевая функция: минимизируем пиковую заболеваемость и смертность
function cost_multi(x)
    # x[1]:  $\beta_{und}$ , x[2]: death_rate, x[3]: detection_time
    model = initialize_sir(;
        Ns = [1000, 1000, 1000],
         $\beta_{und}$  = fill(x[1], 3),
         $\beta_{det}$  = fill(x[1]/10, 3),
        infection_period = 14,
        detection_time = round{Int, x[2]},
        death_rate = x[3],
        reinfection_probability = 0.1,
        Is = [0, 0, 1],
        seed = 42,
        n_steps = 100,
    )

    infected_frac(model) = count(a.status == :I for a in allagents(model)) / nagents(model)
    dead_count(model) = 3000 - nagents(model)

    peak_infected = 0.0

    # Запускаем с несколькими повторами
    replicates = 5
    peak_vals = Float64[]
    dead_vals = Int[]

    for rep = 1:replicates
        model = initialize_sir(;
            Ns = [1000, 1000, 1000],
             $\beta_{und}$  = fill(x[1], 3),
             $\beta_{det}$  = fill(x[1]/10, 3),
            infection_period = 14,
            detection_time = round{Int, x[2]},

```


Многокритериальная оптимизация параметров

- Создание файла `scripts/sir_optimize_parameters.jl` и запустим его.

```
Optimization stopped after 1 steps and 889.28 seconds  
Termination reason: Max time (120.0 s) reached  
Steps per second = 0.00  
Function evals per second = 0.06  
Improvements/step = NaN  
Total function evaluations = 51
```

```
Best candidate found: [0.122956, 5.34903, 0.0237749]
```

```
Fitness: (0.00067, 0.00000) agg=0.00067
```

Оптимальные параметры:

$\beta_{und} = 0.12295579898158256$

Время выявления = 5 дней

Смертность = 0.023774934501594132

Достигнутые показатели:

- Вывод в консоль: оптимальные параметры и соответствующие им значения критериев.

- Этот скрипт загружает результаты параметрического сканирования (файл `data/beta_scan_all.csv`, созданный `scan_beta.jl`) и строит единый составной график, объединяющий три панели:

— пик эпидемии и конечная доля инфицированных; — число умерших; — доля выздоровевших.

Сводная визуализация результатов

График сохраняется как `plots/comprehensive_analysis.png`. Скрипт предназначен для итогового отчёта и не требует повторного запуска симуляций.



The screenshot shows a Jupyter Notebook window titled "sir_visualize_dynamics.jl". The interface includes a top bar with the file name and a breadcrumb path: `~/work/study/2025-2026/simulation-modeling/study_2025-2026/simulation-modeling/labs/lab04/project/scripts`. Below the bar, there are tabs for several files: `sir_model.jl`, `sir_run_basic.jl`, `sir_scan_beta.jl`, `sir_migration_effect.jl`, `sir_optimize_parameters.jl`, and the active tab, `sir_visualize_dynamics.jl`. The main area displays the following Julia code:

```
using DrWatson
@quickactivate "project"
using Agents, DataFrames, Plots, CSV

include(scrdir("sir_model.jl"))

# Загружаем результаты сканирования
df = CSV.read(datadir("beta_scan_all.csv"), DataFrame)

# Создаём составной график
p1 = plot(df.beta, df.peak, label = "Пик", xlabel = "β", ylabel = "Доля инфицированных")
plot!(p1, df.beta, df.final_inf, label = "Конечная")
p2 = plot(df.beta, df.deaths, xlabel = "β", ylabel = "Число умерших")
p3 = plot(df.beta, df.final_rec, xlabel = "β", ylabel = "Доля выздоровевших")
plot(p1, p2, p3, layout = (3, 1), size = (800, 900))
```

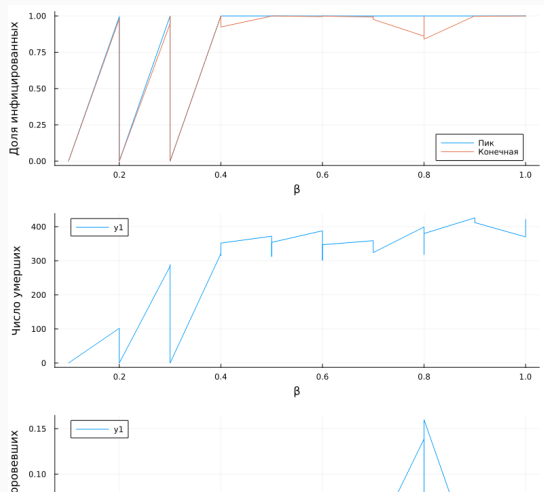
Рис. 13: Рис

- График: `plots/comprehensive_analysis.png` (рис. 4.4). Три панели, позволяющие одновременно оценить:

— порог возникновения эпидемии (рост пика); — нелинейное увеличение смертности; — насыщение доли переболевших.

Сводная визуализация результатов

- В консоль может выводиться дополнительная статистика (например, значения β , при которых пик превышает 5%).



- Я выполнила лабораторную работу.